A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js

Mateusz Miotk[a,*]

[a] *Faculty of Mathematics, Physics and Informatics, University of Gdańsk, Wita Stwosza 57, 80-308, Gdańsk, Poland*

Abstract

Context: Node.js/Express services reach relational databases through a spectrum of access layers (native drivers, query builders, and object-relational mappers, or ORMs), yet practitioners choose between them largely on vendor benchmarks whose numbers are rarely shown to be comparable — fragmented, PostgreSQL-only, and seldom reporting the tail (p99). **Objective:** We contribute a reusable *comparability protocol* for access-layer benchmarking — a semantic-equivalence correctness gate, a documentation-selected estimand paired with a same-SQL standardized contrast, and separated operating points — realized in an open harness and demonstrated through a dual-engine PostgreSQL/MySQL case study. The protocol, not any ranking, is the contribution. **Methods:** An identical Express application is served through nine documentation-selected access layers and two tuned native baselines, over five access patterns, on the latest stable versions (July 2026 freeze). We report throughput with the *high-load response-time p99 under equal closed-loop demand* (a fixed 50-connection point) from 25 repeated runs per cell, and on the deep fetch distinguish it from the tail at *equal utilization* (an open-loop sweep, where the gap converges). An automated cross-check verifies byte-identical responses before measurement. **Results:** The implementation-and-strategy difference concentrates in relation-materializing patterns: the deep/nested fetch

*Corresponding author.
Email address: mateusz.miotk@ug.edu.pl (Mateusz Miotk)

spreads 7.15× (PostgreSQL) and 4.96× (MySQL) between native and the slowest ORM, against only 1.68× and 2.01× among the raw paths running common SQL — a standardized contrast that isolates no single mechanism. The read ordering holds across engines (Spearman $\rho \geq 0.86$), but the aggregation and insert orderings reverse: Prisma drops to last on the MySQL insert, and the per-layer engine advantage scatters 1.6–3.2×. No layer exceeds ≈110% of one core; the native driver leads all ten engine-pattern combinations. **Conclusion:** The case study shows why the protocol matters: access-layer rankings are configuration-specific and version-sensitive, so the durable contribution is the protocol and its harness, not any single ranking.

Keywords: benchmarking methodology, database access layer, object-relational mapping, Node.js, response-time tail (p99), empirical software engineering

---

1. Introduction

Express.js has long been among the most widely used web frameworks for Node.js, and almost every non-trivial service built on it eventually reaches a relational database through some *access layer*. The choice is not settled by the language or the framework: access layers trade raw performance for ergonomics, compile-time type safety, and protection from pitfalls such as the N+1 query problem, where a naive nested fetch silently issues one query per parent row instead of one for the whole result graph [1], a pervasive anti-pattern in real-world database-backed applications (Section 2). Native drivers expose the wire protocol directly (`pg` for PostgreSQL, `mysql2` for MySQL); query builders assemble SQL programmatically (Knex); and object-relational mappers (ORMs) map rows to objects and manage relationships automatically, following patterns such as Data Mapper and Active Record [2]. Node.js back-end performance under load is itself a recognized empirical topic [3, 4, 5], but that literature treats the runtime and web framework as the unit of comparison rather than varying the database access layer.

In practice, the most visible performance comparisons are *vendor benchmarks*: plentiful but narrow, published by the maintainers of the compared products, confined almost exclusively to PostgreSQL, and settling on a single metric family rather than reporting throughput and the tail together. In the sources searched, we did not identify one reporting the p99 tail behavior widely held to matter for user-perceived latency under load [6, 7]. The peer-reviewed literature is sparser still, limited to at most three ORMs, confined to PostgreSQL, and, where it benchmarks PostgreSQL against MySQL directly, holding the access layer fixed rather than varying it [8, 9, 10, 11, 12] (Section 2 surveys each).

In the sources we searched (Section 2) this leaves a gap along the four coverage axes of Table 1 — access-layer taxonomy, engine, joint throughput/tail metrics, and vendor involvement: no access-layer comparison we found includes MySQL, and no PostgreSQL/MySQL benchmark varies the access layer; no independently authored study spans the taxonomy broadly (the broad-coverage comparisons are vendor-authored, academic work covering at most three libraries [10, 11]); and we did not identify one reporting throughput and the p99 tail jointly (the sole vendor suite stops at a single P95 at one load point [13]). Beyond these four axes, no study we found measures client-observed performance through an HTTP framework, academic studies instead instrumenting query execution inside the process [11, 14].

These are gaps in *coverage*. A more basic gap is methodological: across the sources searched, the reported numbers are not established to be *comparable*. A layer that returns fewer fields, or errors faster than it does real work, can appear faster; the query strategy chosen for a library is conflated with the library's own cost; and a single, arbitrary load point conflates capacity, tail latency under demand, and per-request latency. Comparing access layers meaningfully requires establishing that comparability first — the problem this paper takes up.

This paper addresses these gaps with a *comparability protocol* and an independently authored case study that applies it (Section 3 defines the term): an *identical* Express application served through all nine access layers (two native

drivers, Knex, and six ORMs: Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM) over five representative access patterns rather than a measured production workload (point read, keyset range scan, deep/nested fetch, per-author aggregation, and insert; Section 3). Seven layers run against *both* engines under an identical schema, dataset, and pool size; two tuned native-driver baselines mark what hand-tuned driver use achieves. Every (layer, engine, pattern) combination reports throughput (req/s) and the *high-load response-time p99 under equal closed-loop demand* at the fixed 50-connection operating point, from 25 repeated runs. At this near-saturation point the p99 largely tracks capacity and queueing, so a lower-capacity layer looks worse on the tail even when its intrinsic latency is not; we therefore also report the tail at *matched utilization*, where it converges — the more informative view of tail behavior. The study separates two estimands: the *documentation-selected implementation-and-strategy* effect of each layer (RQ1–RQ3), and a controlled *same-SQL* standardized contrast that reports the residual difference once every layer executes identical SQL — changing query formulation, protocol, preparation, round-trips, and mapping together rather than isolating any one. The full harness (adapters, seed data, an autocannon-driven [15] load runner, and an automated cross-check that all layers return *byte-identical* responses on the four non-mutating patterns) is released as an open replication package.[1]

The case study answers three research questions, each scoped to the benchmarked implementations under the specified operating point:

RQ1 *(performance difference):* How large is the throughput and response-time-p99 *difference* of the benchmarked implementations relative to the native-driver baseline, and how does it vary across the five access patterns? We keep distinct three quantities a fixed operating point conflates: *capacity* (saturating throughput), *high-load latency* (p99 under equal demand),

[1] Harness, deterministic seed, all adapters, raw per-cell measurements, and the table/figure-generating scripts: `https://github.com/mmiotk/express-db-access-performance`; release v1.11.7 is permanently archived at Zenodo, `https://doi.org/10.5281/zenodo.21487096`.

and *latency at matched utilization* (equal fractions of each layer's own capacity).

RQ2 *(engine dependence):* Does the observed layer ordering transfer across PostgreSQL and MySQL, or is it engine-dependent?

RQ3 *(consequential patterns):* For which access patterns is the choice of implementation most consequential in this configuration?

This paper's primary contribution is a **comparability protocol for access-layer benchmarking**, specified normatively — inputs, mandatory stages, cell-admission criteria, output interpretation, and applicability limits — in Section 3.1: a set of controls that turn the vendor-benchmark genre — which reports throughput or latency numbers whose comparability is assumed — into a controlled experiment, by establishing three preconditions the genre otherwise leaves unmet.

- **Semantic equivalence.** Every layer must return *byte-identical* results on the protocol's conformance suite (fixed probes plus a seeded property-based input sweep), and every write must produce the intended database state — validated field values, generated identifiers, exact row-count changes, and transactional rollback, not merely a 2xx — before its timing is admitted, so a layer that silently returns less, writes wrong, or errors faster than it works, cannot appear faster (Section 3).

- **Strategy attribution.** The treatment is fixed by a predeclared, reproducible selection rule (*documentation-selected* here; the protocol privileges none), and a *same-SQL standardized contrast* reports how much of an observed difference remains once the SQL is held fixed — a diagnostic contrast, not an attribution to the library's execution machinery versus the query strategy.

- **Operating-point separation.** Capacity, high-load tail under equal demand, and tail at matched utilization are reported as *distinct* quantities, not conflated at one load point.

These preconditions are not new to benchmarking in general: correctness, workload equivalence, warm-up, coordinated-omission correction, capacity curves, and reproducibility are long established in the performance-evaluation literature (Section 2) [16, 7, 17]. The contribution is their *domain-specific synthesis and operationalization* for the access-layer genre — which assumes comparability rather than establishing it — not the invention of these principles. We distinguish this contribution from two things it is easily confused with. The **artifact** — an open, reusable harness (a uniform adapter contract, a deterministic seed, a randomized-block matrix runner, the automated correctness cross-check, and a coordinated-omission-corrected open-loop generator) — *operationalizes* the protocol and is released so it can be re-run, but is not itself the scientific claim (Section 3). The **empirical case study** — eleven implementations (nine documentation-selected layers and two tuned baselines) across PostgreSQL and MySQL over five access patterns (Section 4) — *demonstrates* the protocol and shows why each precondition is load-bearing: a broken MikroORM cell briefly *led* the MySQL write ranking until the equivalence gate excluded it, the sevenfold documentation-selected deep-fetch spread was only 1.7× among the raw paths running common SQL, and a large high-load tail gap dissolved once layers were compared at matched utilization. The case study thus contributes novel experimental evidence — a dual-engine, throughput-and-p99, documentation-selected access-layer comparison we did not identify in the searched sources — but the resulting rankings are configuration-specific and version-sensitive, so the benchmark infrastructure — the protocol's domain-specific synthesis of established benchmarking principles, and its harness — not any single ranking, is the more durable contribution. The case study also yields a practitioner-oriented checklist of genre-specific benchmarking pitfalls (Section 5), each an instance of a precondition the protocol enforces.

2. Related Work

We survey prior work along four *coverage* axes — taxonomy coverage, engine coverage, joint throughput/tail reporting, and vendor involvement — along which access-layer benchmarks vary and which our case study spans (Table 1); more fundamentally, as Section 2 argues, prior work leaves the comparability of its numbers unestablished, the gap our protocol addresses. We located it with a structured scoping search (not a systematic review) of the ACM Digital Library, IEEE Xplore, Scopus, and Google Scholar (July 2026), crossing an access-layer set (*ORM*, "object-relational mapping", "query builder", "database access layer", and the seven library names) with Node.js/Express/ JavaScript, engine, and measurement terms over 2006–July 2026, followed by backward and forward citation chaining. We retained empirical comparisons of relational access layers or engines and excluded non-empirical, non-relational, and single-library studies; the exact strings, per-source dates, and screening decisions are archived with the replication package (`notes/related-work-search.md`). Because this is a scoping rather than exhaustive search, we scope every resulting gap claim to the sources searched.

2.1. Object-relational mapping and the access-layer spectrum

Access layers bridge the object-relational impedance mismatch between the relational model and the object graphs an application manipulates [18], spanning native drivers, query builders, and ORMs following patterns such as Data Mapper and Active Record [2] (Section 1). Torres et al. [19] survey two decades of ORM patterns; the trade-off they document — developer productivity against a hard-to-predict runtime cost — is the one this paper quantifies. The same question recurs in the polyglot-persistence debate over non-relational stores [20], a choice we hold fixed while varying only the access layer.

2.2. ORM performance and data-access anti-patterns

A parallel line studies ORM performance from the opposite direction, detecting and repairing the inefficient SQL that mapping frameworks emit rather

than benchmarking layers. The costliest pattern is the N+1 query problem (Section 1); it and related redundant-query anti-patterns are pervasive across real-world web applications [21, 22, 23], and their performance impact has been quantified in ORM-backed systems [24, 1] and shown often *removable* by query synthesis, round-trip batching, or caching [25, 26, 27] (the JavaScript analogue being mis-sequenced `async`/`await` serializing independent I/O [28]). Closest to our design, Lorenz et al. [29] show the best O/R mapping strategy depends on the database technology, van Zyl et al. [30] that ORM performance is highly tuning-sensitive, and a recent study adds an energy dimension [31]. This literature establishes *where* ORM overhead concentrates (relation-materializing, N+1-prone access) and that it is configurable rather than fixed, which our per-pattern results, in this configuration, are consistent with; it does not compare drivers, query builders, and ORMs head-to-head under controlled load, our aim here.

2.3. Peer-reviewed access-layer comparisons

The peer-reviewed literature that measures access layers head-to-head is narrow. Colley et al. [8] give the earliest systematic cross-language account, benchmarking eight ORM frameworks across Java, C#, Python, and PHP against a common schema, but *deliberately excludes JavaScript*, leaving Node.js, and Express in particular, without an academic baseline for the better part of a decade. Three recent studies partially address this: one, in *Procedia Computer Science*, compares Prisma against optimized raw SQL over eight query patterns on a containerized PostgreSQL setup [9] (cited for its methodology; differing library versions, hardware, workload, and harness mean we neither build on nor dispute its absolute speedup figures); a second compares Prisma, TypeORM, and Sequelize across four relation types, also on PostgreSQL, by response time, memory, and CPU [10]; and the most recent instruments the same three ORMs on PostgreSQL inside a NestJS service, timing internal query stages rather than client-observed requests [11], independently observing the concurrency-dependent Prisma ranking flip we also report (worst at single queries, best under parallel load). A fur-

ther Express-based study optimizes a single stack (pooling, batching, caching) rather than comparing access layers [14].

2.4. Database-engine, SQL/NoSQL, and Node.js performance

A second, largely disjoint line benchmarks the layers immediately below and above the access layer without varying it directly. Salunke and Ouda [12] compare PostgreSQL and MySQL head-to-head under standard OLTP workloads (the only dual-engine precedent found), but benchmark the *engines themselves*, bypassing any application-level access layer, so their work cannot say how a driver, query builder, or ORM would change it; work contrasting SQL and NoSQL databases [32] likewise measures the store, not the access layer. On the Node.js side, Chaniotis et al. [3] established Node's viability as a web platform, Kuffel and Walter [4] tracked how its back-end performance drifts under sustained load, and do Carmo et al. [5] compared back-end frameworks more broadly. None treats the database access layer as an independent variable, let alone varies it across the taxonomy studied here.

2.5. Standard benchmarks and benchmarking methodology

Standard benchmarks define how database systems are measured: YCSB [33] for cloud-serving stores and the TPC-C/TPC-E suites for OLTP. These target the *engine* and its transaction mix, saying nothing about the driver, query builder, or ORM this study varies, but they set the methodological bar for controlled, repeatable load. A separate methodological literature explains why the vendor benchmarks' single-metric habit is itself a problem: Fruth et al. [7] and Raasveldt et al. [16] catalog database-benchmarking pitfalls (coordinated omission, warm-up, tool-induced distortion) that silently corrupt reported percentiles; Dean and Barroso [6] show why the tail, not the mean, determines user-perceived latency once a request fans out, as a deep/nested fetch does; and Li et al. [34] trace its hardware, OS, and application sources. Load-testing surveys report such measurement is often ad hoc [35, 36] and systems results frequently hard to reproduce [37], motivating a shared, reusable harness built

for relevance, repeatability, and fairness [38, 39]. From this literature we take a design choice absent from the access-layer benchmarks surveyed above: report throughput *and* tail latency (p99) for the same run, rather than a single metric family chosen after the fact.

2.6. Practitioner and vendor benchmarks

A larger body of evidence comes from practitioners and vendors, each authored by a party with a stake in one of the compared products. Drizzle publishes both a Northwind micro-suite spanning nine targets (`pg`, Knex, Kysely, MikroORM, TypeORM, Prisma, and Drizzle itself, pooled and unpooled) and an official k6-driven load test reporting requests per second, average latency, P95, and CPU [13]. Prisma's own site compares Prisma, Drizzle, and TypeORM by median query latency over 500 iterations, reporting neither throughput nor any latency percentile [40]. IMDBench, maintained by Gel Data (formerly EdgeDB), adds Sequelize and node-postgres and reports both throughput and latency on three CRUD-style operations [41]. Together these three cover more of the taxonomy than the entire peer-reviewed literature combined, yet share the defects noted in Section 1: vendor authorship, PostgreSQL-only coverage, and a single reported metric family.

2.7. Positioning

In the sources searched through July 2026 (protocol archived with the replication package), the access-layer benchmarks report throughput or latency numbers whose comparability is assumed rather than established, and we did not identify a study combining broad taxonomy coverage, both engines, joint throughput/tail reporting, and vendor involvement — each we found covers at most two or three of these four properties (Table 1). We therefore state the strongest novelty claim the search supports — that we did not identify a prior study combining these properties in the documented search — rather than a claim of priority the scoping search cannot establish. Our case study spans all four, but the paper's contribution is the *comparability protocol* it

applies (Section 1): the semantic-equivalence gate, the same-SQL standardized contrast, and the separated operating points that make such a comparison a controlled experiment and that the genre otherwise leaves unaddressed — closing the engine gap that separates access-layer studies from engine-only work such as [12].

Table 1: Prior work on relational-database access-layer performance, positioned along four *coverage* axes our case study spans; the paper's contribution is the comparability protocol of Section 1, not this coverage. "Layers covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer. *Vendor involvement* marks maintainer-authored studies — a conflict-of-interest property of the source, not a methodological guarantee that an independently authored benchmark is unbiased.

Study	Layers covered	Engines	Metrics	Vendor involvement
Colley et al. [8]	8 ORMs, 4 languages (no JS)	not stated	query latency	None
Procedia CS [9]†	Prisma vs. raw SQL	PostgreSQL	latency, CPU	None
JCCT [10]	3 ORMs	PostgreSQL	latency, memory, CPU	None
JCSI [11]	3 ORMs	PostgreSQL	per-stage latency	None
Salunke and Ouda [12]	none (engine only)	PostgreSQL + MySQL	throughput, latency	None
Drizzle [13]	9 (broad)	PostgreSQL	throughput, latency, P95	Vendor
Prisma [40]	3 ORMs	PostgreSQL	median latency only	Vendor
IMDBench [41]	4 ORMs + driver	PostgreSQL	throughput, latency	Vendor
This study	**9 documentation-selected (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + response-time p99**	**None**

†Cited for its methodology only; its absolute speedup figures are not comparable to ours given the differing versions, hardware, workload, and harness.

3. Study Design

The case study's goal, in Goal–Question–Metric terms, is to *analyze* the relational database access layer of an Express.js service *for the purpose of* quantifying the performance of its documentation-selected implementation strategy *with respect to* throughput and response-time p99 *from the point of view of* an application developer choosing a layer *in the context of* PostgreSQL and MySQL back ends [42]. This section specifies the study's *comparability protocol* (Section 1) — the semantic-equivalence cross-check, the documentation-selected estimand with its same-SQL control, and the separated operating conditions — and the dual-engine case study that applies it. The design follows established

guidelines for controlled experimentation in software engineering [43, 44], crossing seven portable access layers with both engines and all five patterns, plus two engine-specific native drivers and their tuned counterparts `pg-tuned` and `mysql2-tuned` (eleven layers: nine documentation-selected, two tuned reference baselines; full rationale in `METHODOLOGY.md`). The comparison is read at three levels, kept separate throughout. The **primary comparison** (RQ1–RQ3, Section 4) is each layer's *documentation-selected implementation and query/loading strategy* — the API each library's documentation presents first, not a claim about the most common or most performant one. A **same-SQL (raw-path) standardized contrast** then reports what spread remains once every layer executes identical SQL through its raw facility. The **mechanisms** that differ within a layer — eager-loading strategy, wire protocol, round-trips, statement preparation, hydration — change together and are *not* separately or causally identified; the same-SQL contrast is a standardized diagnostic, not a decomposition and not a bound on the intrinsic library effect (Table 5 enumerates the components it moves).

A fixed concurrency imposes equal *demand* on layers of unequal throughput and therefore drives them to unequal *utilization*, so RQ1 is characterized under three operating conditions rather than one privileged point — the protocol's *operating-point separation*. *Equal external demand* is the fixed 50-connection *high-load* operating point — the full five-pattern, two-engine matrix. To fix the relative utilization at that point rather than leave it unknown, a per-pattern concurrency sweep (connection ladder 1–200 for every layer on both engines; Supplement Table S35) locates each pattern's throughput knee: it lies at or below 50 connections for all five patterns, so the fixed point places every pattern at high utilization of its own capacity (median 97–100% across layers on both engines), while 50 connections against a ten-connection pool keep about forty requests queued regardless of pattern (Controls, below). Capacity is thus *measured* for all five patterns (the deep-fetch concurrency curve, Supplement Figure S2, is kept for its per-layer detail). The two utilization-dependent conditions, which need a per-layer *saturating-throughput* target, are demonstrated on

the deep fetch: *equal utilization* is the coordinated-omission-corrected open-loop sweep, each layer offered a fixed fraction (50/70/85/95%) of its *own* saturating throughput, and an *equal compute budget* is the exploratory equal-CPU check. These answer different questions and need not agree, so we report all three.

3.1. The comparability protocol

We state the protocol here independently of the case study; the remaining subsections instantiate it for the access-layer setting. It turns a set of *treatments* compared on a fixed *workload* into a controlled experiment; the box below states its inputs, its mandatory and recommended stages, the cell-admission rule, and the output interpretation compactly, and the remaining subsections instantiate them for the access-layer setting.

The comparability protocol.

Inputs. Treatments (implementations under comparison); a *declared workload model* (stated access patterns, not a production trace); an *output-equivalence semantics*; and an *operating-point definition* separating capacity, external demand, and utilization.

Stages. *Mandatory* (a comparison is valid only if both hold): (1) *correctness oracle* — admit a treatment only if its outputs are equivalent under the semantics and its mutations produce the intended state; (2) *treatment-definition rule* — fix each treatment's implementation and strategy by a rule declared reproducibly in advance; the protocol requires such a rule but privileges none (this case study's is documentation-first, with a tie-breaker; Study Design below). *Recommended* (they enrich the comparison and each may be scoped to a representative workload point rather than the whole matrix): (3) *strategy control* — add a standardized contrast holding the query strategy fixed, read against a common-strategy baseline; (4) *capacity identification* — locate saturating throughput by a concurrency sweep; (5) *demand and utilization* — measure the tail at equal external demand and at matched fractions of each treatment's own capacity.

Cell admission. Time a (treatment, workload-point) cell only if the oracle passes — equivalent non-mutating outputs and a correct post-write state, not merely a success status; non-equivalent output, an invalid or unsuccessful write, or a degenerate plan (e.g. N+1) disqualifies it.

Outputs. Throughput and tail latency per operating point, *descriptive* of the treatment as defined — the standardized contrast reports a residual, not a causal decomposition — with rankings configuration- and version-specific and only *relative* within-condition differences meant to travel.

Applicability limits. the oracle must match the output semantics. Byte-identical comparison, used here, is valid only when equivalent outputs are deterministic and canonically serializable. Where they are not (unordered collections, floats, timestamps, nondeterministic identifiers, or differently serialized equivalents), byte-equality is inappropriate and the oracle must instead be a semantic comparator (set equality, numeric tolerance, or canonicalization). The case study's outputs are deterministic and canonically constructed, so byte-equality is a valid

oracle for it.

Compliance levels. An instantiation need not exercise every stage on every cell. We name five *compliance levels* — the *mandatory validity core*, *capacity characterization*, and the *standardized-strategy*, *utilization-controlled*, and *resource-normalized* extensions. Here the first two hold across all five patterns on both engines, while the three extensions are demonstrated on the deep fetch — the most consequential pattern — as a representative-point instantiation, not full-matrix coverage; Supplement Table S40 maps each cell to the levels it satisfies.

Table 2 makes the protocol's contribution explicit: each row pairs a generic benchmarking principle with its access-layer manifestation, the failure that follows if the stage is skipped, and the evidence that the stage was load-bearing here. We do not invent these principles, but in the sources searched found no prior access-layer benchmark applying all five together; the stages are complete (admission through interpretation), individually necessary (omission changes the conclusion), and distinguishable, yet transferable — Supplement Table S37 shows analytically how each prior benchmark's conclusion becomes uninterpretable once its absent stages are named.

3.2. Factors and treatments

We vary three factors. The *access layer* has eleven levels across three tiers (schematic below): two native drivers with two engine-specific tuned baselines (`pg-tuned` reuses named prepared statements, `mysql2-tuned` the binary protocol via `execute()`), the query builder Knex, and six ORMs. The tuned baselines are reference points, not documentation-selected treatments, leaving *nine* documentation-selected layers.

Table 2: The comparability protocol, stage by stage. Each row pairs a generic benchmarking principle with its concrete manifestation for relational database access layers, the failure that follows if the stage is omitted, and the evidence that the stage was load-bearing in this study. We do not claim to originate these principles; in the sources searched we found no prior access-layer benchmark applying all five together. Throughputs abbreviated PG (PostgreSQL) and MySQL; "conns" denotes client connections.

Stage	**Benchmark decision**	**Generic principle**	**Access-layer manifestation**	**Failure if omitted**	**Evidence it was load-bearing**
Correctness oracle	Which treatments are timed at all	Validity precedes speed; never time incorrect results	Layers differ in output shape and mutation semantics; a "fast" cell may be a silent error doing less work, whereas engines return one canonical result	Broken/incomplete layers look fastest and top the ranking	Broken MikroORM MySQL write led the insert ranking until the write-state gate excluded it (silent 500s); a read cross-check caught a fan-out aggregation bug and a driver result-shape mismatch
Treatment-definition rule	Which API and loading strategy of each layer is measured	Pre-register and fix the system-under-test configuration reproducibly	One library exposes many equally-official query paths with different round-trip counts; a pure engine has one query language, so no selection rule is needed	Arbitrary/cherry-picked API choice; spread reflects the tester, not the defaults	Documentation-selected deep fetch spreads 7.15× (PG) / 4.96× (MySQL); round-trips differ 1–4 by design; Drizzle is the documented tie-break case
Strategy control	Separate query-strategy effect from raw-execution effect	Hold confounds constant via a common-baseline contrast	Access-layer time mixes loading strategy with execution machinery; engines have no ORM loading strategy to confound, so no same-SQL baseline is meaningful	Attribute strategy-driven spread to library "overhead"	The 7.15×/4.96× spread narrows to 1.68× (PG) / 2.01× (MySQL) among raw paths on common SQL, so most spread is query strategy, not raw execution
Capacity identification	Locate each layer's saturating throughput before fixing an operating point	Interpret latency relative to capacity, not at an arbitrary fixed load	Layers saturate at very different throughputs, so a fixed connection count lands them at different capacity fractions; a single engine saturates near one throughput	A fixed load compares layers at unequal utilization	A per-pattern sweep (1–200) puts every knee at or below 50 conns; median utilization 97–100% at the 50-conn point; native ∼7× the slowest ORM's capacity
Demand & utilization	Report tail at equal demand and at matched own-capacity fractions, as distinct quantities	Distinguish demand from utilization; correct coordinated omission	Because layer capacities differ, an equal-demand tail conflates queueing with intrinsic latency; engines at similar capacity make the two nearly coincide	Read a capacity/queueing gap as an intrinsic latency difference	At 50 conns deep-fetch p99 runs pg 20 ms to MikroORM 116 ms; at 50% of each layer's own capacity the tails converge to ∼2–5 ms, so the gap is capacity-and-queueing, not intrinsic latency

Tier	Implementations	Runs on
Native driver	`pg`, `mysql2`	its own engine
Tuned native baseline	`pg-tuned`, `mysql2-tuned`	its own engine
Query builder	Knex	both engines
ORM	Drizzle, Prisma, Sequelize, TypeORM, Objection, MikroORM	both engines

In total, 11 implementations = 9 documentation-selected (2 native + 1 query builder + 6 ORMs) + 2 tuned; the 7 portable layers run on both engines and the 4 native/tuned are engine-specific, so $4 + 7 \times 2 = 18$ layer×engine cells × 5 patterns = 90 measured configurations. We classify a library by its dominant interface, not its self-description (Section 1); Objection, for instance, layers an ORM over Knex. The nine documentation-selected layers are a representative cross-section of the three tiers, not an exhaustive catalogue — each maintained, used, and, for the portable tiers, running against both engines — which excludes PostgreSQL-only Slonik and pg-promise as non-portable and Kysely as a query builder already covered by Knex (`METHODOLOGY.md` gives the full criteria). Vendor independence means authorship only: no evaluated library's maintainers were involved, though the consequential author choices (selection, schema, pool size, tuning) remain and are disclosed. The *engine* has two levels, PostgreSQL 18.4 and MySQL 9.7.1, and the *access pattern* has five, realized as HTTP endpoints (Supplement Table S39), giving the 90 measured configurations counted in the schematic above. Supplement Table S29 records which factors are held constant, vary, or are uncontrolled on this single virtualized host — the reason we report results for this configuration rather than as general library performance.

3.3. Objects: schema, workload, and data

The workload is a minimal blog domain (`authors` 1−∗ `posts` 1−∗ `comments` ∗−1 `authors`), exercising a one-to-many relationship and a two-hop join for the deep fetch. The five endpoints implement five representative access patterns [33] (Supplement Table S39): the keyset page uses `WHERE id < cursor ORDER BY`

`id DESC LIMIT` n against the primary-key index rather than a large `OFFSET`; the deep/nested fetch returns a post with its author and every comment, each with its own comment-author; and the aggregation reports post count, comment count, and total views per author. The database is loaded from a deterministic seed (2,000 authors, 100,000 posts, 1,000,000 comments) by the native driver, independently of any layer under test, so every engine receives the same seed *data payload* (byte-identical row values); on-disk representation, index layout, and collation naturally differ between engines. The working set fits in memory, so the measurements emphasize the access layer's per-request instruction, protocol, and mapping cost with disk-read latency largely removed, while engine-side execution, buffer management, locking, and logging remain part of every measurement [45]. Every request draws its target identifier uniformly at random from the full seeded range (posts 1–100,000, authors 1–2,000), so load spreads across the working set rather than one hot record; a skewed, production-like key distribution is a planned variant (Section 6).

3.4. The adapter contract and N+1 control

Every access layer implements the same factory interface (five query methods plus a teardown), so the Express server and load runner are layer-agnostic and each returns an *identical result shape*. Each uses that layer's *documentation-selected* mechanism to avoid the N+1 query problem on the deep fetch — a hand-written join for the native drivers, Knex, and the query-builder-style Drizzle (the tuned baselines reuse the native join), and relation eager loading (`include`, `withGraphFetched`, `populate`, or `relations`) for the data-mapper ORMs. The comparison is therefore one of each layer's *documentation-selected configuration* under a fixed adapter-selection rule, not a fixed query plan: a measured difference reflects the SQL each layer generates, its round-trip count, and how it materializes results, with one asymmetry inherent to any driver-versus-ORM comparison (the native default is hand-authored SQL, the ORM default the library's relation loader). Server-side statement logging captures the exact SQL, round-trip count, and `EXPLAIN` plans (released with the package); the per-

endpoint counts differ by design (Supplement Table S2: the deep fetch takes one query in Sequelize and MikroORM, two in the native drivers, Knex, Drizzle, and TypeORM, and four in Prisma and Objection). The documentation-selected API was fixed by a rule decided *before* measurement — the eager-loading API each layer's official documentation presents first for the pinned version, breaking ties (where several official paths are equally prominent, as for Drizzle's SQL-style builder versus its higher-level relational-query API) toward the API at the library's own taxonomy tier and recording the rest as documented alternatives — a reproducible selection heuristic, not evidence that the chosen API is performance-optimal or the most common production choice, so we report a documentation-selected implementation-and-strategy estimand that models the choice of a developer following the official documentation — a construct whose corroboration and limits are set out in the supplement's Construct-validity section (Table S33) and Section 6. This documentation-first rule is the case study's *treatment-selection policy* — one instance of the protocol's mandatory predeclared, reproducible rule (Section 3.1), which privileges no particular rule; a study targeting expert-tuned usage would substitute its own. Supplement Table S16 and `METHODOLOGY.md` record, per layer, that API with its documentation page and version, its round-trip count, the documented alternative we did *not* use, and that query logging, hooks, and validation are off.

To standardize the SQL and contrast the combined contribution of query strategy and formulation — the protocol's *strategy-attribution* precondition — every adapter also implements a *same-SQL standardized contrast*: identical two-statement deep-fetch SQL (and, per engine, identical `EXPLAIN` plans) with identical row mapping, executed through the layer's raw-SQL facility, mirroring the aggregation control (Section 4). Server-side capture, synchronized to a request marker so connection handshakes are excluded, confirms the two control statements are byte-identical across every layer on both engines; what differs is the wire protocol each driver negotiates (per-layer protocols in Supplement Table S14). The six ORMs split both ways on the join-versus-select-in default. Because the native driver's documentation-selected default is a hand-authored

join while an ORM's is whatever relation loader its documentation presents first (the driver-versus-ORM asymmetry noted above), the deep fetch — the one pattern exposing a documented strategy choice — admits two *co-primary* regimes, which we measure on one harness and report side by side (Section 4, Table 6). The *documentation-primary* regime is the strategy fixed by the adapter-selection rule above. The *performance-conscious* regime applies a predefined *tuning budget*: the faster of a layer's two documented deep-fetch loading strategies (join versus select-in), admitted only where it is byte-identical to the documentation-primary output under the same `bench/verify.mjs` oracle. The budget is deliberately narrow — it tunes only the loading strategy, only on the deep fetch, and never rewrites SQL, adds caching, or changes the schema (Section 4 reports the resulting regimes). To ensure no adapter silently returns less or issues N+1 queries, the protocol's *semantic-equivalence* precondition is established at four levels rather than asserted as a universal property: (i) *specification*, shared canonical constructors that fix each response's field set, key order, and serialization; (ii) *finite pre-measurement conformance*, a fast gate (`bench/verify.mjs`) asserting full JSON byte-equality to the native-driver baseline on 12 fixed probes per adapter, both engines, before any timing; (iii) *exhaustive* equivalence, infeasible for arbitrary code and not claimed; and (iv) *property-based* randomized differential testing (`bench/verify-property.mjs`), which asserts byte-equality over a fixed-seed sample of thousands of random and edge-case inputs, on both engines, with *zero* divergences (Supplement Table S38). Byte-equality thus remains a *finite*, if much broadened, check — strong evidence of, not proof of, agreement on every possible input. For the mutating patterns a companion check (`bench/verify-writes.mjs`) validates database *state*, not the HTTP status alone: through an independent native-driver connection it confirms, for every adapter on both engines, that the single-row insert wrote exactly the field values and generated identifier requested, that the transactional write inserted the post and its five comments with the exact field values (one `posts` and five `comments` rows), and that a transaction whose last comment violates the author foreign key throws and leaves no partial state (atomic rollback); all adapters pass

on both engines. The read cross-check caught two defects during development (a fan-out aggregation bug and a driver result-shape mismatch; Section 5).

3.5. Controls

Four controls hold the shared conditions fixed — connection pool, logical task, physical dataset state, and observer — so that the *configured access-layer implementation bundle* is the only *deliberately varied* factor; they do not isolate the library alone: the bundled differences in SQL, query count, protocol, loading strategy, and hydration are the treatment (Table 5), not confounds to be removed. *(i) Pool size* is fixed at ten connections for every adapter, so the comparison reflects the access-layer choice, not pool tuning [46, 47] (for Prisma, whose default pool is core-count-derived, this is pinned via `connection_limit`). Because the load generator opens 50 concurrent connections against this ten-connection pool, roughly forty in-flight requests are always waiting in each library's acquisition queue; this queueing is deliberately part of what we measure and is held identical across layers (a 200 ms pool sampler confirmed the native PostgreSQL cell ran with its pool fully occupied and about 32 requests pending). *(ii) Identical query semantics*: each endpoint issues the same logical query across layers, aggregation using a single correlated-subquery formulation everywhere (the documented raw-SQL path), so the residual reflects the layer on an identical plan, not the generated SQL. *(iii) Write isolation*: because the insert endpoint mutates the dataset, every cell resets it before warm-up and before every measured run, and the write endpoint is measured in its own dedicated boot after the database's physical state is rebuilt (PostgreSQL: recreate from a seeded template; MySQL: rebuild with `OPTIMIZE TABLE` and re-pin `AUTO_INCREMENT`), so physical layout and purge state cannot drift across replicates or leak between adapters. *(iv) Observer separation*: the HTTP load generator runs in a process separate from the server.

3.6. Measurement procedure

Load was generated with `autocannon` [15], a closed-loop (constant-concurrency) HTTP/1.1 generator [48] issuing requests at fixed concurrency

and recording a latency histogram. Each of the 90 configurations was measured as 25 *repeated runs*: for each replicate we booted a fresh server process, opened 50 concurrent connections, ran a fifteen-second warm-up whose measurements were discarded — long enough for every layer to reach and sustain at least 95% of steady-state throughput, since managed runtimes do not always stabilize promptly [49] — then took one twelve-second measured run. Booting a fresh process per replicate prevents persistent application-process state (JIT, pool, heap) from carrying across replicates; all runs share one host and one short campaign, so replicate index and measurement order are treated as blocking factors, and cell order was reshuffled independently within each replicate (a forward-versus-reversed order check found no history effect on the read cells; supplement, *Measurement details*). Within a replicate, every layer and engine is driven by the identical request-id sequence from a PRNG seeded on the (endpoint, replicate) pair, so this paired-stream design removes between-layer sampling noise as a confound. The experimental unit is the freshly booted server run for one (layer, engine, endpoint) cell in one replicate; individual HTTP requests are subsamples. We report the median of the 25 replicate values of throughput and of the run-level percentiles p50, p90, p97.5, and p99 (one cell, Drizzle's MySQL insert, has 24 after a failed health check; the median and interval accommodate it). Throughout, *p99* denotes this *median run-level p99* — the median across replicates of each run's own p99 — not the p99 of the pooled request distribution across runs, which the design does not estimate. A 60 s re-measurement of the deep fetch preserves the p99 ranking exactly and barely moves each cell's p99 (Supplement Table S23), so the 12 s window is adequate for the tail. During measured runs we sampled server CPU and peak resident memory over the full process tree, with database and load-generator CPU sampled separately (supplement, *Measurement details*). Inserts were measured under each engine's *default* durability configuration (PostgreSQL `fsync` and `synchronous_commit` on; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`), the regime a deployment runs unless an operator relaxes it; a labelled secondary run relaxed all of those settings to isolate the durability

mechanism (Supplement Table S3). To characterize behavior under load we swept the deep/nested fetch across 1–256 connections for every layer and engine [50], with an exploratory equal-CPU-budget check (Section 4); and a native `undici`-based constant-arrival generator drove the deep fetch under a fixed offered rate (250–4,000 req/s) rather than fixed concurrency, measuring latency from each request's intended send time and clipping timed-out requests at their cutoff, a coordinated-omission-corrected design [51, 52] that independently validates the closed-loop tail (Section 4).

3.7. Analysis

We separate *primary* outcomes, which carry the paired inference and the research-question claims, from *secondary* and *exploratory* outcomes, reported descriptively as controls and sensitivity analyses (Supplement Table S34): the primary outcomes are throughput and the closed-loop p99 on the five patterns against both engines at the fixed operating point.

Because absolute throughput is hardware-specific, we analyze *relative* differences within an engine; a pattern's *spread* is the documentation-selected native driver's median throughput divided by that of the slowest layer on that pattern (native-relative, so comparable across patterns). We report medians over the 25 repeated runs, the coefficient of variation as a stability signal [17, 53], and a percentile bootstrap 95% CI for the median from a fixed seed; these intervals capture within-campaign, run-to-run variability on this host and configuration, not variation across machines, versions, or deployments (Section 6). The design is a randomized block — within each replicate every layer runs the identical request stream — so adjacently ranked layers are compared with *paired* tests on their per-replicate throughput ratios (a two-sided sign-flip permutation test cross-checked with the Wilcoxon signed-rank test), reporting the geometric-mean paired ratio, its bootstrap CI, and the paired dominance, plus a TOST against a $\pm 5\%$ margin (author-selected and application-dependent, not a universal threshold; supplement) for the closest pair; we foreground these effect sizes and interval estimates over p-values, as recommended for randomized

performance experiments [54, 55]. Because the seven adjacent pairs follow the *observed* ranking, their significances are a post-hoc, descriptive robustness check, not a confirmatory family. Adjacent-layer p99 gaps of one or two milliseconds sit at the millisecond measurement floor and are not read as real; conclusions rest on the large-ratio patterns, chiefly the deep fetch. The full construction — the permutation and bootstrap procedure and seed, tie handling, the layer×engine interaction test, the rank correlations, and the equivalence-margin rationale — is given in the supplement's *Statistical methods* section; the paired p99 comparisons are Supplement Table S30.

3.8. Experimental setup

The measurements were taken on 2026-07-16 to 2026-07-18 on a single host: the primary matrix ran overnight and the secondary experiments (order-invariance, durability, same-SQL, open-loop, fan-out, equal-CPU, concurrency, utilization, pool-size, cluster, mixed-workload, wait-event, post-restart, and the longer-window tail) over the same window — a virtualized Intel Xeon Gold 6140 instance (16 vCPUs, single NUMA node, 126 GB RAM; Linux 6.17; Node.js 24.18.0; Express 5), with PostgreSQL 18.4 and MySQL 9.7.1 running locally. Each library is pinned to the *latest stable release compatible with the harness adapter contract* as of the freeze date (2026-07-15), recorded from the committed lockfile and an automated environment capture (Supplement Table S11); only Sequelize invokes the earlier-release exception (its version-7 line is a prerelease, so the current 6.x stable line is used). Because access-layer performance is version-sensitive, this pins a dated snapshot rather than a permanent ranking (Section 4).

3.9. Replication package

The harness — the Express application, the adapter contract, all nine documentation-selected adapters and the two tuned baselines, the deterministic seed, the `autocannon` matrix runner, the correctness cross-check, and the scripts regenerating every table in Section 4 — is released so the full matrix

can be re-run with one command against a containerized or local PostgreSQL and MySQL.

4. Results

The tables report throughput (requests per second, higher is better) and tail latency (p99, milliseconds, lower is better) per access layer and engine. The two consequential patterns are in the main text (the deep/nested fetch, Table 3, and the insert, Table 4); the point read, keyset range scan, and aggregation are in Supplement Tables S12, S13, and S15. All numbers come from the run of Section 3. Each native driver is reported both as documentation-selected (`pg`, `mysql2`) and tuned (`pg-tuned`, `mysql2-tuned`), the tuned rows marking the throughput ceiling hand-tuned driver use reaches, not a level every documentation-selected layer attains without code changes. We compare *relative* differences within an engine; two estimands recur below: the *documentation-selected implementation-and-strategy effect* (RQ1–RQ3) — what a developer following each library's documentation obtains, not its intrinsic overhead — and the *same-SQL effect* (Supplement Table S14) — what remains when every layer runs common SQL through its raw facility, a compound standardized contrast that reports the residual spread rather than decomposing or bounding the difference (construct validity: Section 6).

4.1. RQ1: performance difference relative to the native baseline

The native driver leads all ten engine-pattern combinations: on PostgreSQL `pg` (or its tuned counterpart) is fastest on all five patterns, and `mysql2` leads the two reads, the deep fetch, aggregation, and the insert on MySQL. The current-generation ORMs sit mid-to-low throughout, Prisma among the lowest: seventh of eight on both point reads, near the bottom of aggregation on both engines, and last on the MySQL insert (Tables 3, 4; Supplement Tables S12 and S15). This retires an earlier version's headline of Prisma tying the native driver at a large CPU premium (Section 5).

Table 3: Deep/nested fetch: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	3687 [3602–3737]	20 [18–21]	–	–
mysql2	native-driver	–	–	2382 [2357–2451]	28 [28–29]
pg-tuned	native-tuned	3746 [3684–3828]	18 [17–18]	–	–
mysql2-tuned	native-tuned	–	–	2415 [2387–2427]	25 [24–26]
knex	query-builder	2487 [2465–2505]	27 [26–28]	2007 [1989–2031]	30 [29–31]
drizzle	orm	1865 [1837–1878]	35 [32–35]	1318 [1300–1336]	47 [46–48]
prisma	orm	1186 [1160–1203]	51 [51–53]	634 [618–644]	93 [89–95]
sequelize	orm	991 [977–998]	60 [59–61]	886 [868–898]	67 [66–68]
typeorm	orm	1204 [1188–1223]	50 [49–52]	1017 [987–1029]	57 [56–59]
objection	orm	1028 [1020–1037]	57 [56–58]	834 [828–855]	69 [67–71]
mikroorm	orm	516 [499–525]	116 [113–119]	480 [474–488]	120 [118–128]

Table 4: Insert: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within one host and campaign, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6402 [6242–6547]	11 [10–15]	–	–
mysql2	native-driver	–	–	1753 [1717–1763]	113 [109–119]
pg-tuned	native-tuned	6413 [6073–6571]	11 [10–12]	–	–
mysql2-tuned	native-tuned	–	–	1750 [1482–1759]	120 [111–130]
knex	query-builder	4841 [4629–4908]	15 [14–16]	1670 [1470–1692]	127 [117–137]
drizzle	orm	4085 [3928–4146]	15 [14–18]	1730 [1324–1745]	120 [109–138]
prisma	orm	2774 [2750–2815]	23 [22–24]	886 [828–903]	153 [137–160]
sequelize	orm	2732 [2719–2747]	23 [21–25]	1478 [1375–1684]	125 [118–132]
typeorm	orm	2648 [2630–2694]	23 [22–26]	1345 [1301–1359]	125 [119–129]
objection	orm	3907 [3746–3980]	17 [15–23]	1728 [1598–1746]	117 [108–132]
mikroorm	orm	1979 [1949–1997]	30 [29–31]	1257 [1240–1268]	117 [109–125]

`pg-tuned` and `mysql2-tuned` track their documentation-selected counterparts within a few percent on the single-round-trip patterns and edge them only on the deep fetch, where issuing more than one round trip lets reusable prepared statements and MySQL's binary protocol pay off (`pg-tuned` 3,746 vs. `pg` 3,687; `mysql2-tuned` 2,415 vs. `mysql2` 2,382); on PostgreSQL aggregation `pg-tuned` adds about 8% (7,538 vs. 6,978).

On the **point read**, the native-relative spread is 2.78× on PostgreSQL and 2.11× on MySQL (Supplement Table S12), with Prisma next-to-last, ahead of only MikroORM.

On the **deep/nested fetch** (a post, its author, and all comments with their authors), the native-relative spread is the widest measured: 7.15× on PostgreSQL (`pg` 3,687, 95% CI [3,602–3,737], down to MikroORM's 516 [499–525]) and 4.96× on MySQL (`mysql2` 2,382 [2,357–2,451] down to MikroORM's 480 [474–488]). The native driver leads outright on both engines — no tie at the top — and the data-mapper ORMs, especially MikroORM, trail furthest (Table 3). The comparison is paired (Section 3); every adjacent step of the MySQL ranking is large relative to its bootstrap interval (Supplement Table S36).

Aggregation is the least layer-sensitive pattern: the native-relative spread is only 1.83× on PostgreSQL and 1.72× on MySQL (Supplement Table S15), and the native driver leads both. The documentation-selected difference is small when a layer forwards a single query and largest on patterns that materialize a nested object graph application-side, such as the deep fetch.

4.2. Same-SQL control

The *same-SQL control* (Section 3) is a *compound* intervention: replacing each layer's documentation-selected relational-loading path with the same hand-written SQL through its raw facility changes the loading API, query strategy, SQL formulation, round-trip count, statement preparation, and result hydration *together* (Table 5). Server-side capture confirms the statement text is byte-identical across layers on both engines while the wire protocols differ per layer, so the control is *same SQL, protocol disclosed*, not *same plan*. It is therefore a

standardized-SQL contrast: it reports how much of the spread remains once the SQL is held fixed. Because raw mode changes several mechanisms at once and may interact differently with each library, this residual is a diagnostic contrast, not a bound on the intrinsic library effect — no ordering theorem holds without assumptions about interaction effects. It does not attribute the difference to any single component, and in particular does not establish how much comes from SQL formulation or query count.

Table 5: The *same-SQL control* is a compound intervention: switching each layer from its documentation-selected path to the same hand-written SQL through its raw facility moves every component below at once — equalizing the SQL text, query strategy, round-trip count, and result mapping, while the raw-SQL facility, statement preparation, and disclosed wire protocol still differ across layers. The residual difference it leaves is a standardized-SQL contrast, not a bound on their combined effect, and cannot be attributed to any single component — in particular, not to SQL formulation or query count.

Component	Documentation-selected path	Same-SQL (raw-path) control
Loading API	the ORM's eager-load API (`include`, `populate`, ...) or a builder join	the layer's raw-SQL facility
Query strategy	per layer: single join, batched select-in, or multi-query	one hand-written two-statement join, identical across layers
SQL formulation	per-ORM generated statement text	byte-identical statement text (server-side confirmed)
Round-trip count	one to four statements	two statements
Statement preparation	ORM-managed	the raw facility's default
Wire protocol	per driver (extended/simple, text/binary)	per driver, unchanged and disclosed
Result hydration	the ORM's object-graph mapping	shared canonical constructors, identical across layers

Among the raw paths running common SQL, the deep-fetch native-relative

spread is far smaller — 1.68× (PostgreSQL) and 2.01× (MySQL), against the documentation-selected 7.15× and 4.96× (Supplement Table S14). The result this supports is that the documentation-selected paths differ much more than the raw paths executing common SQL; it does *not* quantify how much of the original difference comes from SQL formulation or query count, which change together with the API, protocol, preparation, and hydration (Table 5). Prisma is the slowest layer on the raw path itself; we conjecture this reflects dispatch and marshalling overhead in its raw API rather than SQL execution, a hypothesis the composite contrast cannot confirm. A loading-strategy sensitivity check (Supplement Table S18) argues against a mere default-strategy artifact: switching the join-default ORMs (Sequelize, MikroORM) to select-in makes them slower, whereas Objection's batched-select-in default is about 1.6× slower than a single join on MySQL, so part of *its* deficit there is a strategy choice rather than a fixed cost.

Because the deep fetch is the only pattern exposing a documented strategy choice, we report RQ1 there under *two co-primary regimes* (Section 3): the documentation-primary strategy and a performance-conscious regime taking the faster byte-identical documented strategy (Table 6). The performance-conscious regime shifts the picture only marginally. For the join-default ORMs (Sequelize, MikroORM) the documentation-selected strategy is already the faster one, so the two regimes return the *identical* configuration; the only cell that gains is Objection on MySQL, whose documented single-join alternative beats its select-in default. Even so the performance-conscious ORM deep fetch stays well below the native driver on both engines, so the deep-fetch gap reflects the libraries' documented strategies rather than a poor-default artifact. TypeORM's alternative errors and Objection's diverges from byte-identity on PostgreSQL, so those cells are excluded as non-drop-ins. This makes RQ1's construct explicit: the numbers below report the performance of each layer's *documentation-selected implementation strategy*, and the performance-conscious regime caps how much a developer could recover within a narrow, semantically-equivalent tuning budget.

Table 6: The two co-primary deep-fetch regimes, measured on one harness (fixed post id, a warmup pass then 25 timed repeats each; absolute req/s therefore differ from the seeded-id primary matrix by design, but every cell here shares the harness, including the native reference row). *doc-primary* is each layer's documentation-selected loading strategy; *perf-consc.* is the performance-conscious regime — the faster of the layer's two documented deep-fetch strategies (join versus select-in), admitted only where byte-identical to the documentation-primary output. For Sequelize and MikroORM the documentation-selected join is already the faster strategy, so the two regimes coincide; only Objection on MySQL gains (its documented single-join alternative beats its select-in default), and even then its performance-conscious throughput stays below the native driver. A dash marks a layer×engine cell whose documented alternative is not a byte-identical drop-in in the pinned versions (typeorm/postgres (alt endpoint status 500); objection/postgres (response not byte-identical); typeorm/mysql (alt endpoint status 500)); there the performance-conscious regime coincides with documentation-primary by definition.

		PostgreSQL (req/s)		MySQL (req/s)	
Layer	Documented alt.	doc-primary	perf-consc.	doc-primary	perf-consc.
native (`pg`/`mysql2`)	hand-join (single)	3347	—	2274	—
`sequelize`	join→select-in	894	894	818	818
`mikroorm`	join→select-in	409	409	416	416
`objection`	select-in→join	–	–	801	1262
`typeorm`	join→select-in	–	–	–	–

4.3. RQ2: engine dependence of the ranking

The read ordering is stable across engines, but the aggregation and insert orderings reverse at the top, so we characterize the transfer by these concrete rank reversals and by the layer×engine interaction magnitudes rather than by a correlation coefficient computed over only seven portable layers. On the **insert**, `knex` leads on PostgreSQL (4,841 req/s) but drops to third on MySQL (1,670), where `drizzle` takes the lead (1,730); Prisma falls furthest, from fourth on PostgreSQL (2,774) to last on MySQL (886, behind even MikroORM's 1,257) — a reversal invisible to a single-engine benchmark. On **aggregation** the same shape recurs: `drizzle` leads on PostgreSQL (6,280) but falls to fourth on MySQL (3,520), where `knex` leads (3,842), while Prisma and Objection swap the bottom two places (Supplement Table S27). The interaction is equally visible in *magnitude* (Supplement Table S28, per-layer PostgreSQL÷MySQL throughput ratios): every layer is faster on PostgreSQL, so the interaction lies in the *spread* of that advantage, not its sign. Across the three reads the spread stays within ≈1.0–1.9×, but the insert scatters from 1.6× (MikroORM) to 3.2× (Prisma) and aggregation from 1.4× to 1.9× — a two-to-one range in the per-layer engine advantage on the writes that reorders the layers across engines. The blocked permutation test (Section 3) confirms this interaction is nonzero but bounds only its existence, not its size. For completeness the rank correlations these reversals produce are reported (descriptive only, and coarse at $n = 7$): $\rho = 0.86$, 0.89, 0.86 on the point read, range scan, and deep fetch, and $\rho = 0.64$ ($\tau = 0.52$) on aggregation and $\rho = 0.68$ on the insert — but the finding rests on the reversals and interaction spreads above, not on the coefficients. Single-engine results are thus portable in order for the reads and not for the writes.

On the **keyset range scan** the native driver again leads (native-relative spread 4.21× on PostgreSQL, 3.85× on MySQL; Supplement Table S13), driven disproportionately by MikroORM alone (905 / 856 req/s); the earlier collapse under `OFFSET` pagination was a workload artifact, not an engine or layer property (Section 5).

Inserts are the pattern where the engine dominates most visibly, under each

engine's *default* durability configuration (Section 3). On MySQL the faster layers cluster within about 5% near an engine-imposed floor (`mysql2` 1,753 down to Knex 1,670 req/s; Table 4), with Prisma the exception at 886 (last), widening the nominal MySQL spread to 1.98×. Direct `performance_schema` instrumentation attributes the floor to the commit path: relaxing durability lifts the flush-bound native driver and query builder about 2× on the MySQL insert while the slower ORMs, limited by their own write overhead, gain less (Supplement Tables S3, S19), so the engine's commit path is the floor for the lean layers, not a layer property. PostgreSQL, by contrast, spreads 3.23× (`pg` 6,402 down to MikroORM 1,979) and is only weakly durability-sensitive (≤14%), so there the access layer, not the engine, remains the larger factor.

4.4. RQ3: which patterns matter most

Ranking the five patterns by native-relative spread gives, on PostgreSQL, deep fetch (7.15×) > range scan (4.21×) > insert (3.23×) > point read (2.78×) > aggregation (1.83×); MySQL keeps the same head and tail (deep fetch 4.96×, aggregation 1.72× last) while the insert and point read swap in the middle, the insert compressing because MySQL's engine floor, not the access layer, sets its ceiling (RQ2). The deep/nested fetch is thus the most consequential pattern on both engines and aggregation the least. Because a fixed 50-connection demand could in principle place fast and slow patterns at different fractions of their capacity — confounding this spread ranking — we measured that it does not: a per-pattern concurrency sweep (Supplement Table S35) puts every pattern's throughput knee at or below 50 connections on both engines, so all five sit at high utilization of their own capacity (median 97–100% across layers) and the ranking compares them at comparable, not unknown, relative utilization. The primary deep-fetch workload uses posts with roughly ten comments; a fan-out sweep from 0 to 500 comments shows the native-relative spread is a roughly fixed multiplier with graph size rather than growing with it (exploratory; an earlier growing-spread finding did not replicate).

4.5. Tail latency

We report the high-load p99 under equal closed-loop demand for every cell at the 50-connection point, so the tail includes connection-acquisition queueing — the tail a deployment sees under load. At this high-load point tail latency tracks throughput: on the deep/nested fetch, PostgreSQL's p99 ladder runs from `pg` 20 to MikroORM 116 ms (Table 3), mirroring the throughput ranking; a paired p99 analysis (Supplement Table S30) resolves every adjacent pair on both engines except two near-ties at the millisecond resolution. Each run-level p99 is a noisy top-1% estimate, so we report the median of the 25 run-level values rather than pooling requests (spread in Supplement Figure S4). Driving the deep fetch under the coordinated-omission-corrected open-loop harness (Supplement Tables S6, S17) reproduces this ordering: corrected tails are flat below saturation and collapse beyond each layer's capacity, the data-mapper ORMs earliest. Offering each layer 50/70/85/95% of *its own* saturating throughput (Supplement Tables S21–S22), the tails converge (near 2–5 ms at 50% utilization on both engines), so the large high-load gap (`pg` 20 ms versus MikroORM 116 ms) largely dissolves at matched utilization: it is a capacity-and-queueing effect, not a difference in tail latency at comparable utilization. Table 7 shows both readings per layer, so the false "intrinsically worse tail" reading is averted directly.

4.6. Measurement stability and effect sizes

Across the 25 repeated runs the coefficient of variation of deep-fetch throughput is at most 4.0% on either engine (Supplement Tables S1, S9), well below the between-layer spreads; dispersion is larger only on the insert, where the MySQL cells are visibly bimodal under group-commit batching — structure a coefficient of variation (up to 15.9%) cannot convey — so we read the write dispersion from the replicate plot (Supplement Figure S1) and the bootstrap spread in the insert table, and report the insert rank correlation conservatively. Every measured request succeeded (zero errors or non-2xx across all 90 cells), load-bearing counters since a cell erroring faster than it works can spuriously lead

Table 7: The deep-fetch tail read two ways, per layer and engine. *Equal demand* is the high-load response-time p99 (ms) at the fixed 50-connection closed-loop point — the headline p99 of Table 3, which at this saturating load largely tracks capacity and connection-acquisition queueing. *50% util.* is the coordinated-omission-corrected p99 when each layer is instead offered half of *its own* saturating throughput (Supplement Tables S21–S22). The wide equal-demand ladder (`pg` 20 to `mikroorm` 116 ms on PostgreSQL) collapses to a 2–5 ms band at matched utilization, so the high-load gap is a capacity-and-queueing effect, *not* an intrinsic tail-latency difference; the matched-utilization reading is the more informative one.

		PostgreSQL p99 (ms)		MySQL p99 (ms)	
Layer	Category	equal demand	50% util.	equal demand	50% util.
`pg`	native-driver	20	3.9	–	–
`mysql2`	native-driver	–	–	28	4.2
`knex`	query-builder	27	2.4	30	1.9
`drizzle`	orm	35	3.8	47	3.7
`prisma`	orm	51	4.2	93	5.2
`sequelize`	orm	60	2.5	67	2.5
`typeorm`	orm	50	3.1	57	2.8
`objection`	orm	57	3.1	69	2.9
`mikroorm`	orm	116	3.9	120	4

a ranking (Section 5). We foreground paired effect sizes over significance tests. The confirmatory contrast is *predefined*, not selected from the ranking: against the native-driver reference, fixed before analysis, every portable layer trails the native driver in all 25 replicates on both engines (dominance 1.00; Supplement Table S41). Adjacent ranks separate similarly, all but the narrowest pair (Sequelize over Objection, 1.05) exceeding the run-to-run CV with the faster layer ahead in every replicate. These bootstrap intervals are within-campaign, not deployment-general (Section 6); the adjacent-pair permutation and Wilcoxon tests, which only re-test pairs the ranking selected, are a descriptive supplement check (Supplement Table S36).

4.7. Scaling with concurrency

A concurrency sweep from 1 to 256 connections (Supplement Figure S2) shows the three-band ordering (native driver, then Knex and Drizzle, then the data-mapper ORMs) emerges by 8 connections and holds at every operating point above about 32; at a single connection the layers are far closer together (about 320–1,020 req/s) and Prisma is mid-pack throughout, never approaching the native driver. The 50-connection operating point sits above the knee for every layer on the deep fetch, so the RQ1 ranking is not an artifact of a single point.

4.8. Resource footprint

On the current library generation the application-side CPU cost is uniform (Supplement Tables S5, S10; the trade-off is plotted in Supplement Figure S3): every layer draws about one core on the deep fetch (100–110%), so none buys throughput with extra application cores. What differs is database-side load: consistent with the native `pg` driver's leaner SQL, the native driver leads throughput and end-to-end compute efficiency at once (`mysql2` 1,254 versus Prisma 384 and MikroORM 356 req per combined-CPU-second on the MySQL deep fetch; Supplement Table S5, since the app-tier figure alone excludes database CPU). An exploratory equal-CPU check and a 1-to-4-worker cluster check (Supplement

Tables S4, S24) are consistent with low per-process throughput being addressed by additional application processes until another bottleneck is reached, with no database-side saturation measured. Peak memory ranges 215–356 MB, lowest for the query builder and lightest ORMs (Supplement Table S10) [56].

5. Discussion

Version-sensitivity: the protocol, not any single ranking, is the contribution. An earlier campaign found Prisma (then 5.22, on an in-process Rust query engine) tying the native driver on the deep fetch while drawing about five application cores; re-running the *identical* harness on the latest stable releases at the freeze did not reproduce it — no layer now draws more than about one core, and Prisma has fallen to mid-to-low throughput (Section 4). The non-reproduction is a fact; its cause is a hypothesis (only Prisma changed architecture, so its Rust-free rewrite is the likeliest driver, but the re-freeze moved the whole toolchain at once). The headline is therefore version-sensitive [30, 57] — again the reason the protocol and its harness, not any single ranking, are the durable contribution.

Implications for practice. RQ1 and RQ3 agree that the documentation-selected implementation-and-strategy difference is specific to the access *pattern*, largest in these probes where a layer must hydrate rows into objects and resolve associations rather than merely issue SQL (the object-relational impedance mismatch [18] made concrete): the native-relative spread is widest on the deep/nested fetch and narrowest on aggregation. Round-trip count alone does not set throughput — on the PostgreSQL deep fetch the single-query MikroORM is the slowest of all while the two-query Drizzle outruns the four-query Prisma (Supplement Table S2; Table 3) — consistent with result-graph materialization, not statement count, separating the layers here.

Compute cost and horizontal scaling. Under current versions, with every layer at roughly one application core, the native driver's lead is consistent with its issuing leaner SQL that pushes more work onto the database tier (Section 4;

Supplement Figure S3, app-tier, with combined CPU in Table S5). A low per-process throughput is not a fixed ceiling over the range tested: on PostgreSQL Prisma's aggregate throughput rises roughly in proportion to workers (1,117 to 4,449 over one to four workers), reaching native-like throughput in this run, trading footprint and energy for throughput [31]. This is a bounded 1-to-4-worker check on a single host, not general scalability: on MySQL the native driver keeps scaling and the gap does not close, and it locates no database-side saturation bottleneck. The open-loop cross-run further indicates that a closed-loop benchmark alone would underestimate how abruptly, not just how much, a heavyweight ORM's tail degrades once offered load crosses its capacity.

Writes: a partly engine-bound cost under default durability. RQ2 has a clear answer on inserts, under each engine's *default* durability configuration: the native driver leads on both engines, PostgreSQL's insert throughput stays strongly layer-dependent even with fsync on (3.23×; Table 4), whereas on MySQL the faster layers cluster within about 5% near an engine-level commit floor (the per-commit redo-log-then-binlog double flush [45]; Supplement Table S19), and relaxing durability does not overturn this engine contrast. The write ranking reverses across engines — `knex` leads on PostgreSQL but `drizzle` on MySQL, and Prisma drops from fourth to last (Table 4) — with the per-layer engine advantage spanning 1.6–3.2× (Supplement Table S28), so a single-engine write comparison invites the wrong generalization either way; an engine-only benchmark such as [12] cannot expose this interaction, whereas the dual-engine design does, consistent with earlier work showing database technology modulates mapping-strategy cost [29]. A transactional multi-statement write (a post and five comments in one transaction) narrows the representative-layer spread to about 3.7× (Supplement Table S8), below the widest read spread, because the single commit amortizes the flush across six inserts; Prisma, mid-pack on the single-row insert, falls to the bottom transactionally, so the transactional ordering does not simply track the single-row one.

Methodological lessons: a checklist for access-layer benchmarking. Reaching the figures above meant confronting four confounds that would otherwise have corrupted the comparison silently, which we distill into a reusable checklist (full detail in the supplement's *Benchmarking-pitfalls checklist*): an **aggregation fan-out** (a join evaluated before `SUM`, inflating the aggregate) and a driver result-shape mismatch, both *correctness* bugs the automated byte-equality cross-check caught; **OFFSET pagination** collapsing on MySQL, a pagination-strategy cost easily misread as an access-layer weakness; **write-workload contamination** from an unreset growing table; and a **query-formulation confound** where a mis-scoped pre-aggregation re-scanned the whole table, a query-plan artifact masquerading as an access-layer effect [58]. The write path needs its own gate: MikroORM 7's removal of `persistAndFlush` made every MikroORM insert throw a 500 that returned faster than a real insert, briefly *leading* the MySQL write ranking until the non-2xx counter flagged it. A successful 2xx alone, however, would not catch a *silently* incorrect write, so the write gate also validates post-write database state — inserted field values, generated identifiers, exact row-count changes, and transactional rollback (Section 3) — making database-state validation the write path's correctness gate as byte-equality is the read path's. None of the four is schema-specific; each is a generic failure mode extending the database-benchmarking pitfalls of Fruth et al. [7] and Raasveldt et al. [16] to the access-layer sub-genre, and at least one plausibly explains part of the gap between some vendor-reported and independently verified figures in this space.

Practical guidance. The practical synthesis follows from RQ1–RQ3 and applies to the benchmarked implementations in this configuration, workload, and host rather than to access-layer categories in general. The native driver leads throughput on every pattern on both engines, and the documentation-selected ORM spread was largest on the one deep/nested-fetch shape tested and much smaller on the read-simple patterns and aggregation. This locates the difference among five probes on one synthetic schema, not across relation-materializing workloads

in general: the fan-out sweep (Section 4) varies breadth (0–500 children) but not depth, schema, or query mix, so this is a benchmark observation to be re-measured on the target workload, not an established law. Where a service's hot path is dominated by deep or nested fetches the access layer is consequential and should be benchmarked for the specific application: the tested thin paths (native driver and Knex) led here *for throughput and response-time p99*, but whether that generalizes is a hypothesis for broader study. Where the workload is read-simple or write-bound (especially on MySQL, whose commit floor dominates layer differences under default durability) the throughput cost is small, and a layer's low per-process throughput may be addressed by additional application processes until another bottleneck (database contention or connection limits) is reached. These are performance findings only: the decision also turns on correctness, maintainability, security, type safety, and developer productivity, which this study does not measure and which may outweigh the differences reported here; the recommendations rest on the single host, default durability, and pinned versions of Section 6, and the same-SQL results are a standardized diagnostic contrast on the documentation-selected cost, not a licence to use raw SQL in practice.

6. Threats to Validity

We organize the threats along the four standard validity categories [59, 43].

Construct validity of the measures. Our dependent variables are HTTP-level throughput (req/s) and tail latency (p99) at the Express endpoint, not query-execution time at the driver or database boundary — deliberate, since it captures the full request round trip a deployed service incurs (Express routing, JSON serialization), though the overhead we attribute to the *access layer* may partly reflect these surrounding costs. We hold them near-constant: every adapter satisfies a documented *adapter contract* and funnels its rows through shared canonical constructors (a fixed field set, key order, and value typing, under `TZ=UTC`), and the automated cross-check (`bench/verify.mjs`)

asserted full JSON byte equality against the native-driver baseline, on both engines, across the four non-mutating patterns before any timing run (Section 3). During development that check caught a defect: PostgreSQL's schema declared `created_at` as a timezone-naive timestamp although the column is `timestamptz`, so Drizzle's PostgreSQL responses carried instants shifted by the host's UTC offset — invisible to throughput and to any coarser equivalence check; it was fixed (`withTimezone: true`) before the campaign reported here. The fixed-payload baseline floor sits well above the fastest measured cell (Section 3), so the framework floor compresses, rather than manufactures, between-layer differences, which are therefore attributable to how each layer reaches an identical response, not to what is returned. Writes are the one endpoint that mutates state; the reported finding uses each engine's *default*, vendor-standard durability configuration (Section 3), not one we relaxed ourselves, so the write spreads (Table 4) describe a standard deployment rather than a tuning choice; a symmetric relaxed-durability secondary confirms the cross-engine gap is not an artifact of either configuration (Section 5, Supplement Table S3).

Construct validity of the treatment. RQ1's treatment is each layer's *documentation-selected* implementation and eager-loading strategy: an operational rule — the API the official documentation presents first — that we do not validate against surveyed, typical, or performance-tuned usage. It is a reproducible proxy for one persona, the developer who follows the official documentation, not a claim that documentation ordering equals idiomatic or performance-conscious practice; where a layer's documentation-first API differs from what a performance-conscious developer would choose, RQ1's spread mixes the library's execution machinery with that strategy choice, a mixture made material by the one-to-four statement range on the deep fetch (Supplement Table S2). Three things constrain it. First, the selected API is in every case the relations mechanism the official documentation presents first — for the ORMs, the library's canonical eager-loading API — not an obscure

one (Supplement Table S33, corroborated against the official documentation recorded per layer in `METHODOLOGY.md`). Second, the *same-SQL control* makes every layer run common SQL through its raw facility, so the raw-path spread (deep fetch 1.68×, against 7.15× documentation-selected; Section 4) is far smaller — a compound *standardized contrast* that reports how much of RQ1's spread narrows once every layer runs common SQL, rather than decomposing or bounding how much comes from the documentation-selected loading strategy as opposed to the library machinery. Third, where the documented alternative is a byte-identical drop-in, a loading-strategy check (Supplement Table S18) shows the documentation-selected default is the *faster* strategy for the join-default ORMs, so their deficit does not appear to be an artifact of a poor default; the one exception, Objection's slower select-in default on MySQL, is disclosed rather than corrected. RQ1 therefore answers what a documentation-following developer obtains, not the library's intrinsic overhead — which this design *contrasts under standardized SQL* rather than *bounds or measures*.

Internal validity. A naively configured ORM can trigger N+1 queries and appear artificially slow for reasons unrelated to its inherent cost; every adapter must use its layer's documentation-selected eager-loading strategy on the deep-fetch endpoint (Section 3). The byte-level correctness cross-check caught two defects before measurement (Section 5). Explicit warm-up phases per cell, set to exceed the slowest layer's cold-start stabilization, absorb JIT, pool-fill, and plan-cache effects, guarding against environmentally induced measurement bias [60]; two further controls rule out non-layer confounds — benchmark rows reset before every cell (physically rebuilt for writes), and one correlated-subquery plan for aggregation across layers. Every layer and engine within a replicate is driven by the identical, PRNG-seeded id sequence, and a forward-versus-reversed cell-order check found no detectable history effect (Section 3), addressing the concern that processing order could confound layer identity with run position. A residual risk is coordinated omission in the load generator, which can under-report tail latency at saturation [7, 51]; a native, coordinated-omission-corrected constant-

arrival cross-run on the deep fetch (Section 3) is consistent with the tail ranking holding under that stricter model (Section 4; Supplement Table S6).

External validity. All measurements come from a single schema, dataset size, and 16-vCPU virtualized host, with a connection pool fixed at ten, against specific engine and library versions (PostgreSQL 18.4, MySQL 9.7.1). The fixed pool is an *equal configured connection limit* comparison; the pool-size frontier (Supplement Tables S7, S25) preserves the ordering from a pool of 1 to 50, and a mixed read/write workload (Supplement Table S26) preserves the read ordering, so the fixed pool does not drive the ranking. Results may not transfer to larger working sets, other pool sizes, or other versions; because library performance ages quickly we pin the latest stable release compatible with the harness (Supplement Table S11); Prisma is measured at 7.8, the Rust-free client [61]. Two choices are disclosed rather than resolved: Sequelize on its stable 6.x line (7.x is prerelease), and Prisma's MySQL path on the MariaDB adapter, as Prisma ships no `mysql2` adapter. Only the *relative* within-engine ordering is intended to travel; absolute req/s reflect this substrate. The memory-resident working set is a hot-cache regime that makes access-layer overhead more visible; as a workload becomes I/O-bound the spreads should compress toward $1\times$. Three same-host concerns do not drive the ranking: a resource-isolation run pinning application, database, and generator to disjoint cores agrees with the shared-host medians within about 2% (ordering unchanged); a multi-worker check (Supplement Table S24) shows the low-per-process layers scale out roughly linearly over one to four workers, so single-process throughput is not the aggregate ceiling, though it locates no database-side saturation bottleneck; and a ten-minute sustained-load check moves throughput by at most $\pm 1.4\%$, so the short runs are not sampling a transient. Crucially, none of these come from an *independent* substrate: they bound only within-host drift (the post-restart re-run moved absolute throughput up to 16% with the ranking intact, Supplement Table S20), not variation across a different host or hypervisor. A substrate change should preserve the *architectural* conclusions — the within-engine relative ordering and the operating-point

separation — while possibly moving absolute req/s, gap magnitudes, and the cross-engine ordering of RQ2; an independent-host replication of the core deep fetch, and longer-horizon drift [4], remain future work. The protocol's stages are experimental-design controls, argued necessary analytically (Table 2, Supplement Table S37), so single-host measurement bounds the case study, not the protocol.

Conclusion validity. Our analysis (Section 3) reports medians with the coefficient of variation and within-campaign bootstrap 95% intervals (repeatability, not population-general) as stability signals and, because the design is a randomized block, compares adjacently ranked layers with *paired* tests on their per-replicate throughput ratios (paired sign-flip permutation, cross-checked with Wilcoxon; Supplement Table S36). Two bounds reinforce the ordering: the widest spreads dwarf the run-to-run CV ($\leq 4.0\%$; Supplement Tables S1, S9), and the ordering holds across the concurrency sweep at ≥ 32 connections (Supplement Figure S2). Rather than assert a specific statistical power (which would need a prospective effect-size model we did not prespecify), we rest the conclusions on effect sizes and interval estimates — foremost the *predefined* native-driver contrasts (Supplement Table S41), with the ranking-selected adjacent-pair permutation tests kept only as a descriptive post-hoc check (Supplement Table S36). Booting a fresh process per replicate prevents persistent process state from carrying across replicates [62], but the replicates share one host and one campaign, so we treat replicate index and measurement order as blocks rather than claiming full independence; more replicates or longer runs would tighten the intervals further. The secondary experiments that cover only a layer subset or few replicates (open-loop tail, equal-CPU, pool-size, transactional write, and cluster checks; Supplement Table S32) are read as *sensitivity evidence* on the tested subset, not as claims about all eleven implementations.

7. Conclusion and Future Work

This paper's contribution is a **comparability protocol** that makes access-layer benchmarking a controlled experiment rather than vendor-style number-reporting: it admits a layer's timing only once the layer returns byte-identical results (semantic equivalence), reports a documentation-selected estimand paired with a same-SQL standardized contrast (strategy attribution), and separates capacity, tail-under-demand, and matched-utilization latency (operating-point separation). An open, reusable harness operationalizes the protocol, and a dual-engine case study — eleven implementations across PostgreSQL and MySQL over five access patterns — demonstrates it (Section 1).

The documentation-selected implementation-and-strategy difference is concentrated across the five tested patterns, not uniform: sharpest for the deep/nested fetch that assembles a nested object graph application-side, and modest for point reads and aggregation (Section 4). Among the raw paths running common SQL the spread is far smaller, but because that contrast changes several mechanisms together, it is a standardized diagnostic that neither bounds nor attributes the difference to any single one (Section 4). On the current library versions no layer's throughput scales with additional application cores and the native driver leads all ten engine-pattern combinations; the read ordering is similar across the two engines while the aggregation and insert orderings reverse (Prisma drops from mid-pack to last on the MySQL insert), a caution against generalizing single-engine results (Section 5). Access-layer rankings can also be version-sensitive: an earlier version's headline did not survive re-running the harness on newer releases (Section 5). Development also surfaced a reusable checklist of benchmarking pitfalls specific to this genre, one of them, a fast-but-wrong write path that passed the read-level cross-check, added by this very re-run (Section 5).

These findings held under a concurrency sweep (1 to 256 connections) and rest on large between-layer effect sizes with non-overlapping bootstrap intervals, the descriptive adjacent-rank permutation tests agreeing (Section 4). Further

extensions would deepen the evidence — a two-machine cross-run to bound observer interference beyond the open-loop check, additional engines and dataset sizes, and a dedicated study of the N+1 penalty and cold-start latency — and the released harness, exercised again by this revision's re-run against Prisma 7, makes them straightforward, addressing the repeatability gap documented in computer-systems research [37].

CRediT authorship contribution statement

Mateusz Miotk: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. The author is not affiliated with, nor funded by, any of the database libraries or vendors evaluated in this study.

Data availability

The complete replication package (benchmark harness, database schema and deterministic seed, all adapter implementations, the byte-level correctness cross-check, raw per-cell measurements, and the scripts that generate every table in this paper) is publicly available at `https://github.com/mmiotk/express-db-access-performance` and permanently archived at Zenodo as release v1.11.7 (`https://doi.org/10.5281/zenodo.21487096`). A reproducibility summary — versions, requirements, the run commands, the time and resources needed, and which results the harness regenerates automatically from the archived raw data — is in Supplement Table S31 and `REPRODUCE.md`.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used a large language model (Claude, Anthropic) in order to draft and edit the manuscript text and improve its readability and language. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

AI-assisted research software

Separately from the writing above, a large language model (Claude, Anthropic) also assisted the author in implementing and analyzing the benchmark harness. This is research tooling for which the author takes full responsibility, and all AI-assisted analytical code is independently inspectable and verified: the statistical estimators carry unit tests against hand-computed values and known closed-form properties (`bench/stats.test.mjs`, run by `npm test`), every reported table cell traces to a raw per-cell record through a committed generator (`MANIFEST.md`), and the byte-level correctness cross-check independently confirms that all layers return identical responses before any timing is trusted.

References

[1] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Detecting performance anti-patterns for applications developed using object-relational mapping, in: Proceedings of the 36th International Conference on Software Engineering (ICSE), ACM, 2014, pp. 1001–1012. doi:10.1145/2568225.2568259.

[2] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, Boston, MA, 2002.

[3] I. K. Chaniotis, K.-I. D. Kyriakou, N. D. Tselikas, Is Node.js a viable option for building modern web applications? a performance evaluation study, Computing 97 (10) (2015) 1023–1044. doi:10.1007/s00607-014-0394-9.

[4] P. Kuffeł, B. Walter, Changes in Node.js server-side application performance characteristics over time under load, in: Advances in Information Systems Development, Vol. 77 of Lecture Notes in Information Systems and Organisation, Springer, 2025, pp. 275–294. doi:10.1007/978-3-031-87880-0_14.

[5] K. X. Carmo, F. Ferreira, E. Figueiredo, Performance evaluation of back-end frameworks: A comparative study, in: Proceedings of the 20th Brazilian Symposium on Information Systems (SBSI), ACM, 2024, pp. 1–9. doi:10.1145/3658271.3658314.

[6] J. Dean, L. A. Barroso, The tail at scale, Commun. ACM 56 (2) (2013) 74–80. doi:10.1145/2408776.2408794.

[7] M. Fruth, S. Scherzinger, W. Mauerer, R. Ramsauer, Tell-tale tail latencies: Pitfalls and perils in database benchmarking, in: Performance Evaluation and Benchmarking (TPCTC 2021), Vol. 13169 of Lecture Notes in Computer Science, Springer, 2022, pp. 119–134. doi:10.1007/978-3-030-94437-7_8.

[8] D. Colley, C. Stanier, M. Asaduzzaman, The impact of object-relational mapping frameworks on relational query performance, in: 2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE), IEEE, 2018, pp. 47–52, java, C#, Python, PHP — deliberately excludes JavaScript/Node. doi:10.1109/iccecome.2018.8659222.

[9] J. C. Yusmita, R. Arya, J. M. Wijaya, K. M. Suryaningrum, R. R. Siswanto, Optimizing database access strategy: A performance analysis comparison

of raw sql and prisma orm, Procedia Comput. Sci. 269 (2025) 1201–1210. doi:10.1016/j.procs.2025.09.061.

[10] J. Attala, C. Khemapatpapan, Comparing the performance of prisma, typeorm, and sequelize on postgresql, J. Comput. Creat. Technol. 3 (2) (2025) 201–213. doi:10.14456/jcct.2025.16.
URL https://so13.tci-thaijo.org/index.php/jcct/article/view/2330

[11] S. Zhadko-Bazilevych, Analysis of ORM framework approaches for Node.js, J. Comput. Sci. Inst. 37 (2025) 426–430. doi:10.35784/jcsi.7951.

[12] S. V. Salunke, A. Ouda, A performance benchmark for the PostgreSQL and MySQL databases, Future Internet 16 (10) (2024) 382. doi:10.3390/fi16100382.

[13] Drizzle Team, Drizzle orm benchmarks, https://orm.drizzle.team/benchmarks, k6, 1M prepared requests, two-machine setup; reports req/s, avg + P95 latency, CPU (accessed 10 July 2026). (2024).

[14] M. D. Imam Sakti, D. Dirgantara, A. K. Ramadhan, M. A. Ibrahim, Optimizing asynchronous performance in Node.js with Express and PostgreSQL, Procedia Comput. Sci. 269 (2025) 172–181. doi:10.1016/j.procs.2025.08.270.

[15] M. Collina, et al., autocannon: fast http/1.1 benchmarking tool for node.js, https://github.com/mcollina/autocannon, version 7.15.0; accessed 10 July 2026. (2023).

[16] M. Raasveldt, P. Holanda, T. Gubner, H. Mühleisen, Fair benchmarking considered difficult: Common pitfalls in database performance testing, in: Proceedings of the Workshop on Testing Database Systems (DBTest), ACM, 2018, pp. 1–6. doi:10.1145/3209950.3209955.

[17] A. Georges, D. Buytaert, L. Eeckhout, Statistically rigorous Java performance evaluation, in: Proceedings of the 22nd ACM SIGPLAN Conference

on Object-Oriented Programming Systems, Languages and Applications (OOPSLA), ACM, 2007, pp. 57–76. doi:10.1145/1297027.1297033.

[18] C. Ireland, D. Bowers, M. Newton, K. Waugh, A classification of object-relational impedance mismatch, in: 2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA), IEEE, 2009, pp. 36–43. doi:10.1109/DBKDA.2009.11.

[19] A. Torres, R. Galante, M. S. Pimenta, A. J. B. Martins, Twenty years of object-relational mapping: A survey on patterns, solutions, and their implications on application design, Inf. Softw. Technol. 82 (2017) 1–18. doi:10.1016/j.infsof.2016.09.009.

[20] P. J. Sadalage, M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence, Addison-Wesley, Upper Saddle River, NJ, 2012.

[21] J. Yang, P. Subramaniam, S. Lu, C. Yan, A. Cheung, How not to structure your database-backed web applications: A study of performance bugs in the wild, in: Proceedings of the 40th International Conference on Software Engineering (ICSE), ACM, 2018, pp. 800–810. doi:10.1145/3180155.3180194.

[22] C. Yan, A. Cheung, J. Yang, S. Lu, Understanding database performance inefficiencies in real-world web applications, in: Proceedings of the 2017 ACM Conference on Information and Knowledge Management (CIKM), ACM, 2017, pp. 1299–1308. doi:10.1145/3132847.3132954.

[23] S. Shao, Z. Qiu, X. Yu, W. Yang, G. Jin, T. Xie, X. Wu, Database-access performance antipatterns in database-backed web applications, in: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2020, pp. 58–69. doi:10.1109/ICSME46990.2020.00016.

[24] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. Nasser, P. Flora, Finding and evaluating the performance impact of redundant data access for applications that are developed using object-relational map-

ping frameworks, IEEE Trans. Softw. Eng. 42 (12) (2016) 1148–1161. doi:10.1109/TSE.2016.2553039.

[25] A. Cheung, A. Solar-Lezama, S. Madden, Optimizing database-backed applications with query synthesis, in: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2013, pp. 3–14. doi:10.1145/2491956.2462180.

[26] A. Cheung, S. Madden, A. Solar-Lezama, Sloth: Being lazy is a virtue (when issuing database queries), in: Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data, ACM, 2014, pp. 931–942. doi:10.1145/2588555.2593672.

[27] T.-H. Chen, W. Shang, A. E. Hassan, M. Nasser, P. Flora, CacheOptimizer: Helping developers configure caching frameworks for Hibernate-based database-centric web applications, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), ACM, 2016, pp. 666–677. doi:10.1145/2950290.2950303.

[28] A. Turcotte, M. D. Shah, M. W. Aldrich, F. Tip, DrAsync: Identifying and visualizing anti-patterns in asynchronous JavaScript, in: Proceedings of the 44th International Conference on Software Engineering (ICSE), ACM, 2022, pp. 774–785. doi:10.1145/3510003.3510097.

[29] M. Lorenz, J.-P. Rudolph, G. Hesse, M. Uflacker, H. Plattner, Object-relational mapping revisited—a quantitative study on the impact of database technology on O/R mapping strategies, in: Proceedings of the 50th Hawaii International Conference on System Sciences (HICSS), 2017. doi:10.24251/HICSS.2017.592.

[30] P. van Zyl, D. G. Kourie, L. Coetzee, A. Boake, The influence of optimisations on the performance of an object relational mapping tool, in: Proceedings of the 2009 Annual Research Conference of the South African Institute of Computer Scientists and Information Technologists (SAICSIT), ACM, 2009, pp. 150–159. doi:10.1145/1632149.1632169.

[31] A. Bonvoisin, C. Quinton, R. Rouvoy, Understanding the performance-energy tradeoffs of object-relational mapping frameworks, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2024, pp. 626–636. doi:10.1109/SANER60148.2024.00069.

[32] Y. Li, S. Manoharan, A performance comparison of SQL and NoSQL databases, in: 2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM), IEEE, 2013, pp. 15–19. doi:10.1109/PACRIM.2013.6625441.

[33] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, R. Sears, Benchmarking cloud serving systems with YCSB, in: Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC), ACM, 2010, pp. 143–154. doi:10.1145/1807128.1807152.

[34] J. Li, N. K. Sharma, D. R. K. Ports, S. D. Gribble, Tales of the tail: Hardware, OS, and application-level sources of tail latency, in: Proceedings of the ACM Symposium on Cloud Computing (SoCC), ACM, 2014, pp. 1–14. doi:10.1145/2670979.2670988.

[35] Z. M. Jiang, A. E. Hassan, A survey on load testing of large-scale software systems, IEEE Trans. Softw. Eng. 41 (11) (2015) 1091–1118. doi:10.1109/TSE.2015.2445340.

[36] P. Leitner, C.-P. Bezemer, An exploratory study of the state of practice of performance testing in Java-based open source projects, in: Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE), ACM, 2017, pp. 373–384. doi:10.1145/3030207.3030213.

[37] C. Collberg, T. A. Proebsting, Repeatability in computer systems research, Commun. ACM 59 (3) (2016) 62–69. doi:10.1145/2812803.

[38] K. Huppler, The art of building a good benchmark, in: Performance Evaluation and Benchmarking (TPCTC 2009), Vol. 5895 of Lecture Notes in

Computer Science, Springer, 2009, pp. 18–30. doi:10.1007/978-3-642-10424-4_3.

[39] S. E. Sim, S. Easterbrook, R. C. Holt, Using benchmarking to advance research: A challenge to software engineering, in: Proceedings of the 25th International Conference on Software Engineering (ICSE), IEEE, 2003, pp. 74–83. doi:10.1109/ICSE.2003.1201189.

[40] Prisma, Query performance benchmarks, `https://benchmarks.prisma.io/`, prisma/Drizzle/TypeORM; median of 500 iterations; no throughput, no p95/p99 (accessed 10 July 2026). (2024).

[41] Gel Data (EdgeDB), Imdbench: Orm benchmarks, `https://github.com/geldata/imdbench`, accessed 10 July 2026. (2024).

[42] V. R. Basili, H. D. Rombach, The TAME project: Towards improvement-oriented software environments, IEEE Trans. Softw. Eng. 14 (6) (1988) 758–773. doi:10.1109/32.6156.

[43] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, 2nd Edition, Springer, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29044-2.

[44] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, IEEE Trans. Softw. Eng. 28 (8) (2002) 721–734. doi:10.1109/TSE.2002.1027796.

[45] S. Harizopoulos, D. J. Abadi, S. Madden, M. Stonebraker, OLTP through the looking glass, and what we found there, in: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, ACM, 2008, pp. 981–992. doi:10.1145/1376616.1376713.

[46] K. Gupta, M. Mathuria, Improving performance of web application approaches using connection pooling, in: 2017 International Conference of

Electronics, Communication and Aerospace Technology (ICECA), IEEE, 2017, pp. 355–358. doi:10.1109/ICECA.2017.8212833.

[47] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, M. Kalinowski, Data management in microservices: State of the practice, challenges, and research directions, Proc. VLDB Endow. 14 (13) (2021) 3348–3361. doi:10.14778/3484224.3484232.

[48] B. Schroeder, A. Wierman, M. Harchol-Balter, Open versus closed: A cautionary tale, in: Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2006. URL https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html

[49] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, L. Tratt, Virtual machine warmup blows hot and cold, Proceedings of the ACM on Programming Languages 1 (OOPSLA) (2017) 1–27. doi:10.1145/3133876.

[50] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, M. Stonebraker, Staring into the abyss: An evaluation of concurrency control with one thousand cores, Proc. VLDB Endow. 8 (3) (2014) 209–220. doi:10.14778/2735508.2735511.

[51] G. Tene, How NOT to measure latency, https://www.infoq.com/presentations/latency-response-time/, conference presentation; origin of the term "coordinated omission" (accessed 10 July 2026). (2015).

[52] B. Krishnamachari, How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results, https://arxiv.org/abs/2605.00428 (2026).

[53] C. Laaber, J. Scheuner, P. Leitner, Software microbenchmarking in the cloud. how bad is it really?, Empir. Softw. Eng. 24 (4) (2019) 2469–2508. doi:10.1007/s10664-019-09681-1.

[54] A. Arcuri, L. Briand, A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering, Softw. Test. Verif. Reliab. 24 (3) (2014) 219–250. doi:10.1002/stvr.1486.

[55] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, Psychol. Bull. 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.

[56] L. M. Vaquero, L. Rodero-Merino, R. Buyya, Dynamically scaling applications in the cloud, ACM SIGCOMM Comput. Commun. Rev. 41 (1) (2011) 45–52. doi:10.1145/1925861.1925869.

[57] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tůma, A. Iosup, Methodological principles for reproducible performance evaluation in cloud computing, IEEE Trans. Softw. Eng. 47 (8) (2021) 1528–1543. doi:10.1109/TSE.2019.2927908.

[58] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proc. VLDB Endow. 9 (3) (2015) 204–215. doi:10.14778/2850583.2850594.

[59] T. D. Cook, D. T. Campbell, Quasi-Experimentation: Design and Analysis Issues for Field Settings, Houghton Mifflin, Boston, MA, 1979.

[60] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, Producing wrong data without doing anything obviously wrong!, in: Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 265–276. doi:10.1145/1508244.1508275.

[61] Prisma, Prisma 7 performance and benchmarks, `https://www.prisma.io/blog/prisma-7-performance-benchmarks`, announces the query-engine rewrite from the Rust engine to a TypeScript client (accessed 13 July 2026). (2025).

[62] T. Kalibera, R. Jones, Rigorous benchmarking in reasonable time, in: Proceedings of the 2013 International Symposium on Memory Management (ISMM), ACM, 2013, pp. 63–74. doi:10.1145/2464157.2464160.